RETO INTERMODULAR – BDA Y SBD

¿Dónde pueden vivir los jóvenes en Euskadi?

Situación de partida

El acceso a la vivienda se ha convertido en uno de los principales problemas de la población joven. El aumento de los precios del alquiler y de la compraventa plantea una pregunta:

¿Puede una persona joven independizarse en Euskadi con su salario?

Una entidad pública os ha encargado desarrollar un sistema de recogida, almacenamiento y análisis de datos que permita estudiar la situación de la vivienda en distintos municipios de Euskadi.

Vuestro trabajo consistirá en obtener datos reales, procesarlos y utilizarlos para responder a esta pregunta con conclusiones fundamentadas.

Objetivos del reto

Debéis desarrollar una solución utilizando **Python** que permita:

1. Recopilar datos reales sobre precios de alquiler o compraventa mediante **API y web scraping**.
2. Obtener información complementaria sobre salarios, población u otros indicadores que consideréis relevantes.
3. Almacenar en **HDFS** los datos originales obtenidos.
4. Utilizar **Apache Kafka** para simular la recepción de nuevos anuncios inmobiliarios.
5. Guardar la información en **MongoDB**.
6. Realizar la limpieza y transformación de los datos utilizando **PyMongo**.
7. Analizar la información mediante consultas y **Aggregation Pipeline** de MongoDB.
8. Presentar los resultados y justificar vuestras conclusiones.

Requisitos técnicos

BDA – Big Data Aplicado

- Extracción de datos mediante API y scraping.
- Almacenamiento de datos en HDFS.
- Creación de un productor y un consumidor Kafka utilizando Python.
- Gestión de datos en formato JSON o CSV.

SBD – Sistemas Big Data

- Diseño de una base de datos MongoDB.
- Inserción y actualización de documentos mediante PyMongo.
- Limpieza de datos incompletos, duplicados o incorrectos.
- Transformaciones mediante operaciones de MongoDB.
- Consultas y agregaciones para obtener indicadores estadísticos.

Todos los programas desarrollados deberán estar escritos en Python.

Entregables

Cada equipo entregará:

1. **Código fuente:** scripts Python organizados y documentados.
2. **Datos:** ficheros originales almacenados en HDFS y base de datos MongoDB.
3. **Arquitectura:** esquema explicativo del recorrido de los datos desde su origen hasta el análisis.
4. **Informe de resultados:** gráficos, tablas y conclusiones.
5. **Demostración:** ejecución del sistema y explicación de las decisiones técnicas.

Resultado final

El objetivo no es simplemente recopilar anuncios de viviendas.

Debéis demostrar, utilizando datos reales, qué municipios de Euskadi ofrecen mejores posibilidades de emancipación para una persona joven y explicar por qué.

Las conclusiones deberán estar respaldadas por los datos analizados.

BDA – Big Data Aplicado

RA1. Gestión de soluciones y almacenamiento

Criterio	0 puntos	3 puntos	7 puntos	10 puntos
Extracción de datos mediante Python	No obtiene datos	Obtiene datos de una fuente manualmente	Automatiza una API o scraping	Automatiza API y scraping
Almacenamiento HDFS	No almacena datos	Crea directorios	Guarda y recupera archivos	Organiza originales y procesados y demuestra su recuperación
Kafka	No funciona	Crea un tópico	Envía mensajes con productor Python	Productor y consumidor Python funcionan y procesan mensajes
Integración	No hay integración	Los componentes funcionan por separado	Integra dos o más componentes	Demuestra el flujo completo hasta MongoDB

SBD – Sistemas Big Data

RA1 — Adquisición, integración y tratamiento de datos

Peso en la 1.ª evaluación: 46,7 %

Criterio	0 puntos	3 puntos	7 puntos	10 puntos
Integración de fuentes	No carga datos en MongoDB	Carga datos de **1 fuente**	Carga datos de **2 fuentes diferentes**	Carga datos de **2 o más fuentes diferentes** y las relaciona mediante al menos **1 campo común**
Construcción del conjunto de datos	No existe una estructura de datos funcional	Guarda todos los datos en una colección sin tratamiento	Utiliza **2 o más colecciones** o documentos anidados/arrays	Utiliza **2 o más colecciones** y demuestra al menos **1 relación, embedding o referencia** entre datos
Limpieza y transformación con PyMongo	No realiza limpieza	Realiza **1 operación** de limpieza o transformación	Realiza al menos **3 operaciones diferentes**	Realiza al menos **4 operaciones diferentes**, incluyendo tratamiento de **nulos/ausentes, duplicados y tipos/formato de datos**
Automatización	El tratamiento se realiza manualmente	Existe algún script Python parcial	Los datos se cargan y transforman mediante scripts Python	Un script o conjunto de scripts permite ejecutar **carga + limpieza + transformación**, sin modificar manualmente los datos en MongoDB

RA3 — Gestión y almacenamiento en MongoDB

Peso en la 1.ª evaluación: 13,3 %

Criterio	0 puntos	3 puntos	7 puntos	10 puntos
Modelado MongoDB	No crea una BD funcional	Crea al menos **1 colección**	Utiliza **2 o más colecciones** con tipos de datos adecuados	Además, utiliza y demuestra al menos **1 decisión de embedding/referencing** y explica por qué la ha utilizado
CRUD con PyMongo	No utiliza PyMongo	Realiza insert	Realiza mediante PyMongo **insert + update + delete + find**	Realiza las 4 operaciones y comprueba mediante código que el resultado es el esperado
Consultas	No realiza consultas	Realiza consultas con igualdad	Utiliza al menos **4 tipos diferentes** entre filtros, comparación, operadores lógicos, proyección, sort, arrays o campos anidados	Realiza al menos **5 consultas útiles para el reto**, incluyendo una con **campos anidados o arrays** y otra combinando **2 o más condiciones**
Aggregation Pipeline	No utiliza aggregate()	Pipeline con $match y/o $group	Crea al menos **2 pipelines** y utiliza **4 etapas diferentes** en total	Crea al menos **3 pipelines que respondan preguntas del reto** y utiliza **5 etapas diferentes**, incluyendo $group y al menos una entre $lookup, $unwind o $facet
Índices y rendimiento	No crea índices	Crea **1 índice**	Crea un índice relacionado con una consulta y ejecuta explain()	Compara mediante explain() la consulta **antes/después o con/sin índice** e identifica COLLSCAN/IXSCAN y totalDocsExamined
Consistencia	No aplica lo trabajado	Solo describe el concepto	Identifica en su proyecto una operación atómica o un caso donde podría necesitarse una transacción	Implementa una **transacción real** si su modelo la necesita; si no la necesita, demuestra mediante código una operación atómica y justifica técnicamente por qué una transacción sería innecesaria

RA4 — Procesamiento, análisis y preparación de resultados

Peso en la 1.ª evaluación: 40 %

Criterio	0 puntos	3 puntos	7 puntos	10 puntos
Datos heterogéneos	No integra datos	Trabaja con una estructura simple	Utiliza arrays, documentos anidados o datos con estructuras diferentes	Integra datos procedentes de fuentes con **estructuras diferentes** y los adapta mediante Python/PyMongo
Automatización del análisis	Los resultados se obtienen manualmente	Ejecuta consultas manuales	Un script PyMongo ejecuta consultas/agregaciones	Un script ejecuta automáticamente **todos los análisis principales** y devuelve resultados preparados para su representación
Indicadores	No calcula indicadores	Calcula **1 indicador**	Calcula al menos **3 indicadores** relacionados con vivienda/emancipación	Calcula al menos **5 indicadores**, combinando información procedente de **2 o más fuentes**
Respuesta al reto	No responde a la pregunta	Presenta datos sin conclusión	Obtiene conclusiones para al menos **2 municipios o zonas**	Compara al menos **3 municipios/zonas** y cada conclusión está vinculada a un indicador calculado con MongoDB
Preparación para visualización	No prepara resultados	Copia resultados manualmente	Genera mediante PyMongo datos utilizables para gráficos	Genera automáticamente mediante PyMongo **al menos 3 conjuntos de resultados diferentes** preparados para representar